Software Security Issues & Prevention Guide

1. SQL Injection (SQLi)

What is SQL Injection and how can we prevent it?

SQL injection occurs when untrusted user input is directly incorporated into a database query, allowing attackers to manipulate SQL commands, access unauthorized information, or modify database records.

Prevention:

- Use parameterized queries and prepared statements.
- Use ORM frameworks instead of dynamically concatenating SQL strings.
- Validate and sanitize user inputs.
- Apply least-privilege permissions to database accounts.
- Avoid exposing database error messages to users.

2. Cross-Site Scripting (XSS)

What is Cross-Site Scripting and how can we prevent it?

XSS occurs when attackers inject malicious JavaScript into a web application, causing it to execute in another user's browser. This can lead to session theft, unauthorized actions, and data exposure.

Prevention:

- Escape and encode user-generated content before rendering.
- Avoid unsafe DOM manipulation methods such as `innerHTML`.
- Use Content Security Policy (CSP).
- Sanitize HTML when accepting rich-text content.
- Use secure frontend frameworks and their default escaping mechanisms.

3. Cross-Site Request Forgery (CSRF)

What is CSRF and how can we prevent it?

CSRF occurs when an attacker tricks an authenticated user into performing unwanted actions on a website without their knowledge. The browser may automatically include the user's authentication cookies with the malicious request.

Prevention:

- Implement CSRF tokens for state-changing requests.
- Configure cookies with the `SameSite` attribute.
- Validate Origin and Referer headers where appropriate.
- Avoid using GET requests for operations that modify data.
- Require re-authentication for highly sensitive operations.

4. Broken Authentication

What is Broken Authentication and how can we prevent it?

Broken authentication occurs when an application has weaknesses in login, logout, password management, or session handling, allowing attackers to compromise user accounts.

Prevention:

- Use strong password policies and secure password hashing such as Argon2id or bcrypt.
- Implement multi-factor authentication (MFA).
- Apply login rate limiting and account abuse protection.
- Use secure session management.
- Implement account lockout or progressive delays where appropriate.
- Never store passwords in plaintext.

5. Broken Access Control

What is Broken Access Control and how can we prevent it?

Broken access control occurs when users can access resources or perform actions beyond their assigned permissions. For example, a regular employee accessing an administrator's information by changing an ID in an API request.

Prevention:

- Implement Role-Based Access Control (RBAC) or Attribute-Based Access Control (ABAC).
- Validate permissions on every protected backend request.
- Enforce resource-level ownership checks.
- Apply the principle of least privilege.
- Never rely exclusively on frontend permission checks.

- Test unauthorized access scenarios.

6. Insecure Direct Object Reference (IDOR)

What is IDOR and how can we prevent it?

IDOR occurs when an application exposes internal object identifiers and fails to verify whether the requesting user has permission to access the referenced resource. Attackers can modify IDs in URLs or API requests to access other users' data.

Prevention:

- Verify resource ownership on every request.
- Apply authorization checks at the database query or service layer.
- Avoid trusting user-provided resource identifiers.
- Use unpredictable identifiers as an additional security measure, not as a replacement for authorization.
- Test access between different user accounts.

7. Server-Side Request Forgery (SSRF)

What is SSRF and how can we prevent it?

SSRF occurs when an attacker manipulates a server into sending requests to unintended internal or external resources. This can expose internal services, cloud metadata endpoints, and sensitive infrastructure.

Prevention:

- Maintain strict allowlists of permitted destinations.
- Block requests to private, loopback, link-local, and internal network addresses unless explicitly required.
- Validate URLs and resolved IP addresses.
- Disable unnecessary URL schemes and redirects.
- Apply network-level outbound access restrictions.
- Protect cloud metadata endpoints.

8. Command Injection

What is Command Injection and how can we prevent it?

Command injection occurs when an application passes untrusted input to operating system commands, allowing attackers to execute unintended system instructions.

Prevention:

- Avoid executing shell commands using user-controlled input.
- Prefer native programming language libraries over shell execution.
- Use safe process execution APIs with separate arguments.
- Validate input against strict allowlists.
- Run application processes with restricted operating system permissions.
- Never construct shell commands through string concatenation.

9. Path Traversal

What is Path Traversal and how can we prevent it?

Path traversal occurs when attackers manipulate file paths using sequences such as `../` to access files outside the application's intended directory. This may expose configuration files, credentials, or sensitive system data.

Prevention:

- Avoid accepting unrestricted file paths from users.
- Normalize and canonicalize file paths.
- Restrict file operations to approved directories.
- Validate file names and extensions.
- Use generated file names instead of trusting uploaded names.
- Apply filesystem permissions and directory isolation.

10. Insecure File Uploads

What are Insecure File Upload vulnerabilities and how can we prevent them?

Insecure file uploads occur when an application accepts malicious or improperly validated files. Attackers may upload executable scripts, malware, oversized files, or files designed to exploit document-processing libraries.

Prevention:

- Validate file extensions, MIME types, and actual file signatures.
- Set file size and upload limits.
- Rename uploaded files using generated identifiers.
- Store uploads outside the publicly executable web directory.
- Scan files for malware where appropriate.
- Restrict executable file types.
- Secure file download endpoints with authorization checks.

11. Insecure Deserialization

What is Insecure Deserialization and how can we prevent it?

Insecure deserialization occurs when an application processes untrusted serialized data without proper validation. Depending on the format and implementation, attackers may manipulate application objects, bypass security controls, or execute malicious code.

Prevention:

- Avoid deserializing untrusted data using unsafe native object serialization.
- Use safer formats such as JSON with strict schema validation.
- Validate data types and expected fields.
- Use allowlists for permitted object types.
- Keep serialization and parsing libraries updated.
- Never trust client-supplied serialized objects.

12. API Security Vulnerabilities

What are common API security vulnerabilities and how can we prevent them?

API vulnerabilities occur when endpoints lack proper authentication, authorization, validation, or traffic controls. Attackers may exploit these weaknesses to access sensitive information, abuse business operations, or disrupt services.

Prevention:

- Authenticate all protected API endpoints.
- Enforce authorization at endpoint and resource levels.
- Validate request bodies, parameters, and headers.
- Implement rate limiting and request throttling.
- Use HTTPS for all API communication.
- Restrict CORS to trusted origins.
- Apply request size limits and pagination limits.
- Maintain API versioning and retire unsupported endpoints.
- Avoid exposing internal stack traces and sensitive response data.

13. Broken Object Property Level Authorization

What is Mass Assignment and how can we prevent it?

Mass assignment occurs when an application automatically maps client-submitted fields to internal objects without restricting which properties can be modified. An attacker may manipulate protected fields such as `role`, `isAdmin`, or `accountStatus`.

Prevention:

- Use explicit request DTOs and input schemas.
- Define allowlisted fields for every operation.
- Never bind request bodies directly to sensitive database models.
- Protect administrative and system-managed properties.
- Validate updates at the backend service layer.

14. Sensitive Data Exposure

What is Sensitive Data Exposure and how can we prevent it?

Sensitive data exposure occurs when confidential information such as passwords, authentication tokens, personal details, financial records, or business information is stored, transmitted, or displayed without adequate protection.

Prevention:

- Encrypt sensitive data in transit using TLS.
- Encrypt sensitive data at rest where appropriate.
- Mask confidential information in application responses.
- Never expose passwords, tokens, or secrets in logs.
- Apply data classification and retention policies.
- Restrict access to sensitive database records.
- Use secure key management systems.

15. Security Misconfiguration

What is Security Misconfiguration and how can we prevent it?

Security misconfiguration occurs when systems, applications, servers, or cloud services use insecure default settings, unnecessary permissions, exposed debug interfaces, or improperly configured security controls.

Prevention:

- Disable debug mode in production.
- Remove default accounts and credentials.
- Disable unused ports, services, and endpoints.
- Configure secure HTTP headers.
- Restrict administrative interfaces.
- Use environment-specific configurations.
- Perform regular configuration audits.
- Apply secure defaults across all environments.

16. Server and Client-Side Secrets Exposure

How can we prevent API keys, passwords, and secrets from being exposed?

Secrets exposure occurs when confidential credentials are accidentally included in source code, frontend bundles, Git repositories, logs, configuration files, or API responses.

Prevention:

- Store secrets in environment variables or dedicated secret managers.
- Never expose private API keys in frontend code.
- Maintain `.gitignore` rules for sensitive files.
- Keep `.env.example` files limited to placeholder values.
- Rotate credentials immediately after suspected exposure.
- Scan repositories for accidentally committed secrets.
- Use separate credentials for development, testing, and production.

17. Man-in-the-Middle (MITM) Attacks

What is a Man-in-the-Middle attack and how can we prevent it?

A MITM attack occurs when an attacker intercepts or modifies communication between two systems without their knowledge. This can expose authentication credentials, confidential information, and API traffic.

Prevention:

- Enforce HTTPS using TLS.
- Use valid and properly verified SSL/TLS certificates.
- Disable outdated protocols and weak cipher suites.
- Implement HSTS for web applications.
- Never disable certificate verification in production.
- Secure internal service-to-service communication.

18. Denial of Service (DoS and DDoS)

What are DoS and DDoS attacks and how can we prevent them?

Denial of Service attacks attempt to make an application or service unavailable by overwhelming it with requests or resource-intensive operations. DDoS attacks use multiple distributed systems to generate malicious traffic.

Prevention:

- Implement rate limiting and traffic throttling.
- Use WAF and DDoS protection services.
- Set request size and timeout limits.
- Apply caching for frequently accessed resources.
- Restrict expensive database operations.
- Monitor unusual traffic patterns.
- Use load balancing and infrastructure-level protection.

19. Brute Force and Credential Stuffing

What are Brute Force and Credential Stuffing attacks and how can we prevent them?

Brute force attacks involve repeatedly attempting passwords until access is obtained. Credential stuffing uses previously leaked username-password combinations to compromise accounts on other services.

Prevention:

- Implement login rate limiting.
- Use MFA.
- Detect suspicious login patterns.
- Prevent commonly used and compromised passwords.
- Apply progressive delays after failed attempts.
- Notify users about suspicious account activity.
- Avoid revealing whether a username or password was incorrect.

20. Race Conditions

What are Race Conditions and how can we prevent them?

Race conditions occur when multiple operations access or modify shared data simultaneously, producing unexpected results. In financial or business applications, this can cause duplicate transactions, incorrect inventory quantities, or inconsistent records.

Prevention:

- Use database transactions.
- Apply appropriate row-level locking.
- Use atomic operations where possible.
- Implement optimistic concurrency control.
- Add unique constraints for duplicate-sensitive operations.
- Use idempotency keys for payment and transaction APIs.
- Test concurrent requests.

21. Business Logic Vulnerabilities

What are Business Logic Vulnerabilities and how can we prevent them?

Business logic vulnerabilities occur when attackers exploit weaknesses in application workflows rather than traditional technical vulnerabilities. Examples include bypassing approval processes, manipulating prices, skipping mandatory steps, or submitting unauthorized transactions.

Prevention:

- Validate business rules on the backend.
- Enforce workflow state transitions.
- Implement maker-checker or approval workflows for sensitive operations.
- Verify transaction ownership and status before execution.
- Prevent duplicate submissions.
- Test unusual and out-of-order workflows.
- Never rely on frontend restrictions to enforce business rules.

22. Insufficient Logging and Monitoring

Why are Logging and Monitoring important for application security?

Insufficient logging and monitoring make it difficult to detect unauthorized access, suspicious activities, data breaches, and system attacks. Without proper audit trails, identifying the root cause of an incident becomes challenging.

Prevention:

- Log authentication and authorization events.
- Maintain audit trails for sensitive operations.
- Monitor failed login attempts and unusual API activity.
- Implement alerts for suspicious behavior.
- Protect logs against unauthorized modification.
- Never log passwords, private keys, or access tokens.
- Establish incident response procedures.

23. Vulnerable and Outdated Dependencies

What are Vulnerable Dependencies and how can we prevent them?

Applications frequently depend on external packages and libraries. Outdated or vulnerable dependencies may contain known security weaknesses that attackers can exploit to compromise the application.

Prevention:

- Regularly update dependencies.
- Use automated dependency vulnerability scanners.
- Review package versions before installation.
- Remove unused packages.
- Maintain dependency lockfiles.
- Monitor security advisories and CVEs.
- Patch critical vulnerabilities promptly.

24. Insecure Database Configuration

What are Database Security Misconfigurations and how can we prevent them?

Database security misconfigurations occur when databases are exposed publicly, use weak credentials, allow excessive permissions, or lack proper access restrictions.

Prevention:

- Never expose production databases directly to the public internet.
- Use strong authentication credentials.
- Restrict database network access.
- Apply role-based database permissions.
- Encrypt database connections.
- Enable backups and recovery mechanisms.
- Regularly audit database access permissions.

25. Container and Infrastructure Security

What are Container Security Vulnerabilities and how can we prevent them?

Container and infrastructure vulnerabilities occur when applications run with excessive privileges, insecure images, exposed services, or misconfigured deployment environments.

Prevention:

- Use trusted and updated base images.
- Avoid running containers as root.
- Scan container images for vulnerabilities.
- Restrict container capabilities and permissions.
- Never store secrets directly in container images.
- Limit exposed ports and network access.
- Keep orchestration platforms and infrastructure updated.

26. Software Supply Chain Attacks

What are Software Supply Chain Attacks and how can we prevent them?

Software supply chain attacks target the development and delivery process by compromising third-party libraries, build tools, package registries, source repositories, or deployment pipelines.

Prevention:

- Use trusted package repositories.
- Verify package integrity and signatures where available.
- Pin and review dependency versions.
- Secure CI/CD pipelines.
- Protect source control with MFA and branch protection.
- Restrict build and deployment permissions.
- Generate and maintain Software Bills of Materials (SBOMs).

27. Insecure Error Handling

What is Insecure Error Handling and how can we prevent it?

Insecure error handling occurs when applications expose internal technical information through error responses. Stack traces, SQL errors, filesystem paths, server configurations, and internal service details can help attackers understand and exploit the system.

Prevention:

- Return generic error messages to external users.
- Log detailed errors securely on the server.
- Use centralized exception handling.
- Never expose stack traces in production.
- Avoid returning database queries or internal configuration details.
- Implement consistent HTTP status codes.

28. Insecure CORS Configuration

What is CORS Misconfiguration and how can we prevent it?

Cross-Origin Resource Sharing (CORS) misconfiguration occurs when a server allows untrusted websites to access resources from another origin. Improper configuration can expose sensitive API responses to unauthorized websites.

Prevention:

- Allow only trusted origins.
- Avoid wildcard origins when credentials are enabled.
- Restrict allowed HTTP methods and headers.
- Validate CORS configuration separately for each environment.
- Never treat CORS as a replacement for authentication or authorization.

29. Weak Cryptographic Implementation

What are Cryptographic Vulnerabilities and how can we prevent them?

Cryptographic vulnerabilities occur when applications use outdated algorithms, weak keys, insecure random number generation, or improper encryption practices. These weaknesses can make protected information easier to recover or manipulate.

Prevention:

- Use established cryptographic libraries.
- Prefer modern algorithms such as AES-256-GCM for authenticated encryption.
- Use Argon2id or bcrypt for password hashing.
- Generate cryptographically secure random values.
- Store encryption keys separately from encrypted data.
- Avoid deprecated algorithms such as MD5, SHA-1 for security-sensitive hashing, and outdated encryption modes.
- Never implement custom cryptographic algorithms.

30. Insecure Session Management

What is Session Hijacking and how can we prevent it?

Session hijacking occurs when attackers obtain or reuse a valid session identifier to impersonate an authenticated user. This may happen through exposed tokens, insecure cookies, XSS, or weak session handling.

Prevention:

- Use HTTPS-only secure cookies.
- Set `HttpOnly` and appropriate `SameSite` attributes.
- Regenerate session identifiers after authentication and privilege changes.
- Set session expiration and idle timeouts.
- Invalidate sessions during logout and password changes.
- Protect session tokens from exposure in URLs, logs, and frontend storage.

General Security Best Practices

The following practices should be applied across every software development project, regardless of the programming language, framework, or architecture.

1. **Secure by Design:** Consider security requirements during system architecture and development rather than after deployment.
2. **Defense in Depth:** Implement multiple independent security layers instead of depending on a single security mechanism.
3. **Least Privilege:** Give users, services, and applications only the permissions they actually need.
4. **Input Validation:** Validate all incoming data at system boundaries, including API requests, file uploads, and database inputs.
5. **Output Encoding:** Encode untrusted data appropriately before displaying or processing it in another context.
6. **Authentication and Authorization:** Verify both the identity of the user and their permission to perform each operation.
7. **Encryption:** Protect confidential information during transmission and storage.
8. **Secure Configuration:** Maintain hardened configurations across development, staging, and production environments.
9. **Regular Security Testing:** Perform static analysis, dependency scanning, dynamic testing, and penetration testing.
10. **Audit and Monitoring:** Maintain security logs and monitor critical application activities.
11. **Backup and Recovery:** Maintain protected backups and periodically test recovery procedures.
12. **Incident Response:** Establish procedures to detect, contain, investigate, and recover from security incidents.